

Retrieval-Augmented Generation (RAG)

Motivation

Large language models encode a vast amount of knowledge in their parameters during pre-training. However, this knowledge is static: it reflects the training data cutoff and cannot be updated without retraining. Models also frequently hallucinate — generating plausible-sounding but factually incorrect information.

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), addresses these limitations by combining a parametric memory (the LLM) with a non-parametric memory (a retrieved document store). At inference time, relevant documents are retrieved and provided to the model as context.

Core Architecture

A RAG pipeline has two main components: a retriever and a generator. The retriever takes a query and returns a set of relevant documents from an external knowledge base. The generator (an LLM) takes the query concatenated with the retrieved documents and produces the final answer.

The knowledge base is typically a collection of text chunks stored as dense vector embeddings in a vector database. The retriever embeds the query and performs approximate nearest neighbor search to find the most similar chunks.

The Retrieval Step

Dense retrieval uses a bi-encoder model: both the query and each document are independently encoded into vectors, and similarity is measured by dot product or cosine similarity. Models like DPR (Dense Passage Retrieval) and sentence-transformers are commonly used as encoders.

Sparse retrieval methods like BM25 use term frequency statistics rather than dense embeddings. They are computationally cheap and often competitive with dense methods for keyword-heavy queries.

Hybrid retrieval combines dense and sparse methods, using techniques like Reciprocal Rank Fusion (RRF) to merge results from both. This often outperforms either method alone.

Chunking Strategy

Documents must be split into chunks before embedding, since embedding models have a token limit (typically 512 tokens) and retrieving entire long documents is inefficient. The chunking strategy significantly affects retrieval quality.

Fixed-size chunking splits documents into chunks of a fixed token count with some overlap. Semantic chunking splits on natural boundaries like paragraphs or sections. Hierarchical chunking indexes both small chunks for precise retrieval and larger parent chunks for richer context.

Chunk overlap (e.g., 10-20% of chunk size) ensures that information at chunk boundaries is not lost and improves recall for queries that span boundaries.

The Generation Step

Once relevant chunks are retrieved, they are inserted into the LLM's prompt as context. A typical prompt template instructs the model to answer the question using only the provided context and to say when the answer cannot be found.

The number of retrieved chunks is limited by the LLM's context window. With modern long-context models (100k+ tokens), more chunks can be included, but longer contexts can dilute the most relevant information — a phenomenon called the 'lost in the middle' problem.

Naive RAG vs. Advanced RAG

Naive RAG follows the basic retrieve-then-generate pipeline. Its weaknesses include poor retrieval for ambiguous queries, redundant or irrelevant chunks, and no mechanism to verify the retrieved information.

Advanced RAG techniques address these weaknesses. Query rewriting reformulates the user query to be more retrieval-friendly. Re-ranking uses a cross-encoder model to re-score retrieved chunks for relevance. Self-RAG has the model decide when to retrieve and evaluate the quality of retrieved passages.

Corrective RAG (CRAG) adds a retrieval evaluator that assesses whether retrieved documents are relevant and triggers web search as a fallback when they are not.

Evaluation

Evaluating RAG systems requires assessing both retrieval and generation quality. Retrieval metrics include recall at k (what fraction of relevant documents are in the top k results) and mean reciprocal rank (MRR).

Generation metrics include faithfulness (does the answer stay grounded in the retrieved context?), answer relevance (does the answer address the question?), and context precision (how much of the retrieved context is actually relevant?).

Frameworks like RAGAS provide automated evaluation pipelines using an LLM-as-judge approach, scoring faithfulness, answer relevance, and context recall without requiring human annotation.

Common Failure Modes

Retrieval failure: the relevant information exists in the knowledge base but is not retrieved, either because the query and document embeddings are not similar enough or because chunking split the relevant information across boundaries.

Context utilization failure: the model ignores or contradicts the retrieved context, falling back on parametric knowledge. This is more common when the retrieved context conflicts with what the model learned during pre-training.

Hallucination in synthesis: even with correct context, the model may generate incorrect information when synthesizing across multiple retrieved passages or when the context is ambiguous.